当代人工智能实验二 实验报告

A* 搜索与 Alpha-Beta 剪枝

GitHub 代码仓库链接：
https://github.com/3217432735/dis/tree/main/experiment2_search_algorithms

姓名	银启行	学号	10245501485
课程	当代人工智能	日期	2026 年 11 月

一、实验目的

完成迷宫寻路、加权地形、极大极小博弈树和宝藏分配四个任务，理解 A* 中 g、h、f 的作用，以及 Alpha-Beta 剪枝的触发条件；通过固定用例和自行设计的对照实验说明启发式、搜索深度与子节点顺序对结果和搜索开销的影响。

二、算法设计与实验设置

A*：状态为当前坐标，g 表示从起点到当前格的实际代价，h 表示到终点的估计代价，优先队列按 f=g+h 从小到大扩展。迷宫题 g 每移动一步加 1；加权题 g 加上“进入下一格”的地形代价，S 和 E 的进入代价按题目约定取 0。记录 parent 用于回溯路径，g_score 防止重复走旧路径。

可采纳启发式：加权地图中设曼哈顿距离为 d，最小正整数地形代价为 c。由于终点 E 不收费，采用 $h=(d-1)c$（$d\geq 1$ 时）不会高估实际代价。示例地图中
$c=1$，因此本题仍可用曼哈顿距离的保守形式；它通常比 h=0 提供更多方向信息。

Alpha-Beta：根节点为 MAX，按层交替选择最大值和最小值，相邻子节点从左到右访问。MAX 更新 α，MIN 更新 β；当 $\alpha \geq \beta$
时，当前节点剩余分支不会改变上层判断，直接剪去。代码同时记录访问节点数和剪枝节点数。

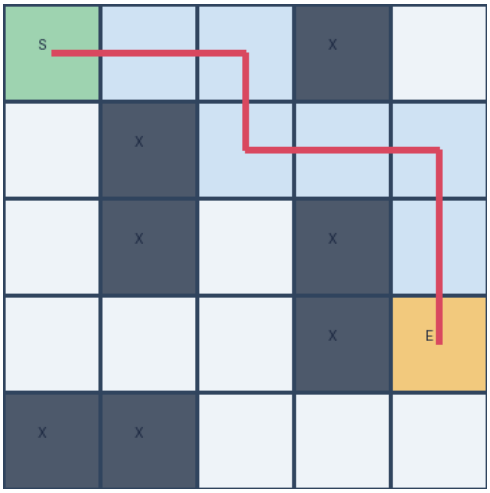
综合题状态：保存 A、B 坐标、剩余宝藏、双方累计收益和当前玩家。任一玩家选宝藏 Ti 时，收益为 Vi-d(p,Ti)；叶节点的评价值为 U=Score_A-Score_B。H=4 时完成四轮回合，H=2 时在 B
第一次行动后评估。

实验使用 Python 3.14、固定地图与固定价值；运行 experiment.py 生成
outputs/results.json，运行 unittest 检查正常路径、无解情况和四个题目的预期结果。

三、题目一：迷宫寻路

固定用例得到的路径为：

(0,0) → (0,1) → (0,2) → (1,2) → (1,3) → (1,4) → (2,4) → (3,4)



图中深灰格为障碍，浅蓝格为最终路径，红线的顺序与程序返回的坐标顺序一致。路径共有 7 次移动；每一步代价都是 1，因此 7 也是最短移动代价。

启发式	展开节点	生成节点	最短步数
h=0	16	17	7
曼哈顿距离	14	16	7

h=0 退化为按 g 搜索，示例中共展开 16 个节点；曼哈顿距离给靠近终点的状态更高优先级，展开数降到 14，路径长度仍为 7。这说明启发式没有改变可行解，只减少了无关状态的扩展。另用 S 与 E 被障碍隔开的地图测试，程序返回 []，未把无解误判为路径。

四、题目二：加权地形

程序找到的最短代价路径：(0, 0) → (1, 0) → (2, 0) → (3, 0) → (3, 1) → (3, 2) → (4, 2) → (4, 3) → (4, 4)

进入节点	地形值	累计 g
(1,0)	1	1
(2,0)	1	2
(3,0)	1	3
(3,1)	1	4
(3,2)	1	5
(4,2)	1	6
(4,3)	1	7
(4,4)=E	0	7

起点本身的代价不计，每次只计算进入下一格的权重。8 和 9 的格子明显增加代价，所以最优解沿左侧和底部绕行。到 E 的最后一步按规则记为 0，最终答案是 7，与题目预期一致；无解输入返回 -1。若简单使用 h=d，在 E 相邻时可能高估 1，因此报告中采用更保守的 (d-1)c，保证启发式可采纳。

五、题目三：Alpha-Beta 剪枝

样例树共有 6 个叶节点，值为 3、5、6、9、1、2。第一组两个 MAX 节点返回 5 和 9，第二组返回 9 和 2；对应两个 MIN 节点为 5 和 2；最后根 MAX 节点取最大值 5。Alpha-Beta 与普通 Minimax 都得到 5，说明剪枝没有改变最优决策。

方法/顺序	结果	访问节点	剪枝节点
Minimax	5	15	0
Alpha-Beta / 原顺序	5	14	1
Alpha-Beta / 较好顺序	5	11	4
Alpha-Beta / 较差顺序	5	15	0

原顺序中，第一棵 MIN 子树已经给出根节点下界 5。访问第二个 MIN 子树时，其第一个叶节点值为 1，已不可能超过 5，因此剩余一个分支被剪去。对照实验表明：较好的子节点顺序在 11 个节点内完成搜索；较差顺序需要访问 15 个节点且没有剪枝。剪枝量主要取决于限制值出现得早晚，而不是节点总数本身。

实现中使用否定式写法切换 MAX/MIN：每层将当前玩家取反，并按需更新 α 或 β。测试还构造了两棵小树，验证叶值、层数和剪枝条件正确，并与 Minimax 的返回结果一致。

六、题目四：竞争环境下的宝藏分配

	A	B	T1	T2	T3	T4
A	0	12	10	4	8	8
B	12	0	8	8	4	10
T1	10	8	0	6	6	12
T2	4	8	6	0	6	6
T3	8	4	6	6	0	6
T4	8	10	12	6	6	0

A* 计算出的 6×6 最小移动代价矩阵如上，A、B、T1-T4 两两之间均可到达。例如 A 到 T2 的距离是 4，首次获得 T2 的即时收益是 6-4=2；但对手随后会争夺剩余宝藏，因此不能只按即时收益选第一步。

深度	算法	最佳首步	最优收益差	访问节点	剪枝分支
2	Minimax	T2	-1	16	0
2	Alpha-Beta	T2	-1	16	0
4	Minimax	T3	3	64	0
4	Alpha-Beta	T3	3	56	4

H=2 时只展开 A 选一次、B 选一次，程序选择 T2，评价值为 -1；两种算法访问相同的16个状态，因为深度较浅，限制值还未形成有效上界。H=4 时能够看到四个宝藏全部分配后的结果，A 的最佳首步变为 T3，最终收益差为 3，与给定预期输出一致。

H=4 的 Minimax 访问 64 个状态；Alpha-Beta 只访问 56 个，并剪去 4 个候选分支，节点数下降约 12.5%。收益相同说明剪枝正确。深度从 2 增加到 4 后决策变化，原因是 H=2 只考虑眼前收益与对手一步回应，而 H=4 能看到之后再两轮的选择，能够避免“先拿高价值但位置变差”的短视行为。

七、自设对照与拓展

自定义 10×10 迷宫	展开节点	生成节点	最终步数
h=0	64	66	18
曼哈顿距离	39	42	18

在图一的基础上，我加入了一条更长的折返路线。h=0 展开 64 个节点，曼哈顿启发式展开 39 个，约减少 39%。地图变大后这种差别比 5×5 样例更明显，因为无启发式搜索容易先扩展大量离终点较远的状态。

我还检查了边界情况：起点等于终点时距离为 0；障碍或无路时返回空路径/-1；Alpha-Beta 与 Minimax 在所有测试树上结果一致；宝藏距离矩阵对角线为 0 且保持对称。9 个单元测试全部通过，固定实验可由脚本重复生成。

八、实验总结

本次实验验证了 A* 通过 g+h 排序减少无效搜索，启发式必须与代价定义匹配；在加权地图中，终点不收费会使得普通曼哈顿距离不再严格可采纳，因此需要更保守的估值。Alpha-Beta 在深层搜索中收益更明显，而好的子节点顺序能更早建立 α / β 边界。综合题中，搜索深度直接改变了 A 的首步选择，说明博弈搜索不能只依赖即时收益。

运行方式：python experiment.py；python -m unittest discover -s tests -v；python build_report.py。